

FINANCE DASHBOARD

Backend API

Technical Documentation

Project	Finance Dashboard Backend
Author	Kunal R. Patil
Technology	Java 17 / Spring Boot 3.4.4
Database	MySQL
Deployment	Railway
Version	1.0.0
Date	April 2026

Live URL: <https://resourceful-youthfulness-production-8c81.up.railway.app>
Swagger UI: </swagger-ui/index.html>

1. Project Overview

1.1 Introduction

Finance Dashboard Backend is a production-grade RESTful API built with Spring Boot 3.4.4 that enables secure management of personal and organizational financial records. It provides endpoints for user authentication, income/expense tracking, and dashboard analytics — all secured with JWT-based authentication and role-based access control.

1.2 Key Features

- JWT Authentication — stateless, token-based security for all protected endpoints
- Role-Based Access Control — three roles (ADMIN, ANALYST, VIEWER) with enforced permissions
- Financial Record Management — create, read, filter, and soft-delete income/expense records
- Dashboard Analytics — total income, total expense, net balance, category summaries, monthly trends
- Pagination & Filtering — paginated record listing with type, category, and date range filters
- BCrypt Password Hashing — secure password storage using BCryptPasswordEncoder
- Soft Delete — records are never permanently removed; marked as deleted with a boolean flag
- Swagger / OpenAPI UI — interactive API documentation with Bearer token support
- Cloud Deployment — deployed and live on Railway with MySQL database

1.3 Technology Stack

Layer	Technology	Details
Language	Java 17	LTS version with modern features
Framework	Spring Boot 3.4.4	Auto-configuration, embedded Tomcat
Security	Spring Security 6.4.4	JWT filter, method-level authorization
Persistence	Spring Data JPA + Hibernate	ORM with MySQL dialect
Database	MySQL 9.1.0	Relational DB, hosted on Railway
Authentication	JWT 0.11.5	JWT token generation and validation
Validation	Jakarta Validation	Bean validation on DTOs
API Docs	SpringDoc OpenAPI 2.8.6	Swagger UI with Bearer auth
Build Tool	Maven	Dependency management and build
Deployment	Railway	Cloud PaaS with GitHub auto-deploy
Code Gen	Lombok 1.18.36	Reduces boilerplate (@Builder, @Data)

2. Architecture & Project Structure

2.1 Layered Architecture

The application follows a clean, layered MVC architecture with a clear separation of concerns between each layer:

Layer	Classes	Responsibility
Controller	AuthController, RecordController, DashboardController	Handles HTTP requests, validates input, delegates to service
Service	RecordService, DashboardService	Business logic, data transformation, orchestration
Repository	UserRepository, FinancialRecordRepository	Database access via JPA, custom JPQL queries
Model	User, FinancialRecord, RecordType, Role	JPA entities mapped to database tables
DTO	LoginRequest/Response, RecordRequest, RegisterRequest, UserResponse	Data transfer objects for request/response
Security	JwtFilter, JwtService	Token validation, authentication context setup
Config	SecurityConfig, CorsConfig, SwaggerConfig	Spring Security, CORS, and OpenAPI configuration
Exception	GlobalExceptionHandler	Centralized error handling with consistent JSON responses

2.2 Request Flow

Every authenticated request follows this flow:

```
HTTP Request
  ↓
Railway Edge (HTTPS termination)
  ↓
Spring Embedded Tomcat
  ↓
CorsConfig → CORS headers applied
  ↓
JwtFilter (OncePerRequestFilter)
  → Extract Bearer token from Authorization header
  → JwtService.validateToken() → HMAC-SHA256 verification
  → Load User from UserRepository
  → Set UsernamePasswordAuthenticationToken in SecurityContext
  ↓
SecurityFilterChain → anyRequest().authenticated() check
  ↓
@PreAuthorize → hasAnyAuthority('ADMIN','ANALYST') check
  ↓
Controller → Service → Repository → MySQL
  ↓
JSON Response
```

2.3 Project Structure

```
finance-dashboard/
├── src/main/java/com/finance/finance_dashboard/
│   ├── FinanceDashboardApplication.java    # Entry point
│   ├── config/
│   │   ├── CorsConfig.java                # CORS mappings
│   │   ├── SecurityConfig.java            # Security filter chain
│   │   └── SwaggerConfig.java             # OpenAPI Bearer auth
│   ├── controller/
│   │   ├── AuthController.java            # Register, Login
│   │   ├── RecordController.java          # CRUD for records
│   │   └── DashboardController.java       # Analytics endpoints
│   ├── dto/
│   │   ├── LoginRequest.java / LoginResponse.java
│   │   ├── RegisterRequest.java / UserResponse.java
│   │   └── RecordRequest.java
│   ├── exception/
│   │   └── GlobalExceptionHandler.java    # @RestControllerAdvice
│   ├── model/
│   │   ├── User.java                     # users table
│   │   ├── FinancialRecord.java          # financial_records table
│   │   ├── RecordType.java              # INCOME | EXPENSE
│   │   └── Role.java                     # ADMIN | ANALYST | VIEWER
│   ├── repo/
│   │   ├── UserRepository.java
│   │   └── FinancialRecordRepository.java # Custom JPQL queries
│   └── security/
│       ├── JwtFilter.java                 # OncePerRequestFilter
│       └── JwtService.java                 # Token gen/validation
├── src/main/resources/
│   └── application.properties             # DB, JWT, JPA config
└── pom.xml                               # Maven dependencies
```

3. Security Implementation

3.1 Authentication Flow

The API uses stateless JWT (JSON Web Token) authentication. No session state is maintained on the server. Every request must carry a valid Bearer token in the Authorization header.

Registration

- Client sends POST /auth/register with name, email, password, and role
- Password is encoded with BCryptPasswordEncoder before saving to database
- Returns UserResponse (id, name, email) — password is never returned

Login

- Client sends POST /auth/login with email and password
- Server loads user from DB, calls passwordEncoder.matches() to verify BCrypt hash
- On success, JwtService generates a signed JWT token (valid 24 hours)
- Token is returned as { "token": "eyJ..." }

3.2 JWT Token Details

Algorithm	HMAC-SHA256 (HS256)
Subject (sub)	User email address
Issued At (iat)	Current timestamp
Expiration (exp)	24 hours from issue time
Signing Key	Configured via JWT_SECRET environment variable (Keys.hmacShaKeyFor)
Header format	Authorization: Bearer <token>

3.3 Role-Based Access Control (RBAC)

Three roles are supported. Permissions are enforced using `@PreAuthorize` annotations with `hasAnyAuthority()` checks (no `ROLE_` prefix — raw authority names are used):

Role	Create Records	View Records	Delete Records
ADMIN	✓ Yes	✓ Yes	✓ Yes
ANALYST	✓ Yes	✓ Yes	✗ No
VIEWER	✗ No	✓ Yes	✗ No

3.4 Security Configuration

Key settings in `SecurityConfig.java`:

- CSRF disabled — appropriate for stateless REST APIs
- Session management set to `STATELESS` — no HTTP session is created
- Public endpoints: `/auth/**`, `/swagger-ui/**`, `/v3/api-docs/**` — no token required
- All other requests require authentication
- `JwtFilter` added before `UsernamePasswordAuthenticationFilter`
- `@EnableMethodSecurity` enables `@PreAuthorize` at method level

4. API Reference

4.1 Base URL

Production	<code>https://resourceful-youthfulness-production-8c81.up.railway.app</code>
Swagger UI	<code>/swagger-ui/index.html</code>

4.2 Complete Endpoint List

Method	Endpoint	Auth Required	Role	Description
POST	<code>/auth/register</code>	No	None	Register a new user account
POST	<code>/auth/login</code>	No	None	Login and receive JWT token
POST	<code>/records</code>	Yes	ADMIN, ANALYST	Create a new financial record
GET	<code>/records?page=0&size=10</code>	Yes	ALL	Get paginated list of records
GET	<code>/records/{id}</code>	Yes	ALL	Get a single record by ID
GET	<code>/records/type?type=INCOME</code>	Yes	ALL	Get records filtered by type
GET	<code>/records/filter</code>	Yes	ALL	Filter by type, category, date range
DELETE	<code>/records/{id}</code>	Yes	ADMIN only	Soft-delete a record
GET	<code>/dashboard/summary</code>	Yes	ALL	Get total income, expense, balance
GET	<code>/dashboard/category-summary</code>	Yes	ALL	Totals grouped by category
GET	<code>/dashboard/recent</code>	Yes	ALL	Last 5 records by creation date
GET	<code>/dashboard/monthly-trend</code>	Yes	ALL	Monthly income vs expense data

4.3 Authentication Endpoints

POST /auth/register

Registers a new user. Password is BCrypt-encoded before storage.

Request Body:

Field	Type	Constraints	Description
name	String	@NotBlank	Full name of the user
email	String	@Email, @NotBlank	Unique email address
password	String	Min 8 chars, uppercase, lowercase, digit	User password (stored as BCrypt hash)
role	Role enum	@NotNull	ADMIN ANALYST VIEWER

Example Request:

```
POST /auth/register
{
  "name": "Kunal Patil",
  "email": "kunal@test.com",
  "password": "Password123",
  "role": "ANALYST"
}
```

Success Response (200):

```
{ "id": 1, "name": "Kunal Patil", "email": "kunal@test.com" }
```

POST /auth/login

Authenticates user credentials and returns a JWT token valid for 24 hours.

Request Body:

Field	Type	Constraints	Description
email	String	Required	Registered email address
password	String	Required	Plain text password (compared with BCrypt hash)

Success Response (200):

```
{ "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ..." }
```

Error Response (400):

```
{ "error": "Invalid password" }
{ "error": "User not found" }
```

4.4 Financial Record Endpoints

POST /records

Creates a new financial record. The authenticated user is automatically set as createdBy.

Required role: ADMIN or ANALYST

Request Body:

Field	Type	Constraints	Description
amount	Double	@NotNull, @Positive	Transaction amount (must be > 0)
type	RecordType	@NotNull	INCOME or EXPENSE
category	String	@NotBlank	Category label (e.g. Salary, Rent)
date	LocalDate	Optional	Date of transaction (yyyy-MM-dd)
description	String	Optional	Free text description

Example Request:

```
POST /records
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
{
  "amount": 5000,
  "type": "INCOME",
  "category": "Salary",
  "date": "2026-04-05",
  "description": "April salary"
}
```

Success Response (200):

```
{
  "id": 1,
  "amount": 5000.0,
  "type": "INCOME",
  "category": "Salary",
  "date": "2026-04-05",
  "description": "April salary",
  "createdAt": "2026-04-05 15:35:11",
  "updatedAt": "2026-04-05 15:35:11",
  "deleted": false
}
```

GET /records?page=0&size=10

Returns a paginated list of all non-deleted records. Required role: Any authenticated user.

Query Parameters:

Field	Type	Constraints	Description
page	int	Required	Page index (0-based)
size	int	Required	Number of records per page

GET /records/filter

Returns filtered and paginated records. All parameters are optional — if none provided, returns all records.

Field	Type	Constraints	Description
type	RecordType	Optional	Filter by INCOME or EXPENSE
category	String	Optional	Filter by category name
startDate	LocalDate	Optional	Start of date range (yyyy-MM-dd)
endDate	LocalDate	Optional	End of date range (yyyy-MM-dd)
page / size	int	Optional	Pagination via Spring Pageable

DELETE /records/{id}

Soft-deletes a record by setting deleted = true. The record remains in the database and is excluded from all queries via repository filters.

Required role: ADMIN only

4.5 Dashboard Analytics Endpoints

GET /dashboard/summary

Returns aggregate financial totals for the current dataset.

Response:

```
{
  "totalIncome": 15000.0,
  "totalExpense": 8500.0,
  "netBalance": 6500.0
}
```

GET /dashboard/category-summary

Returns total amounts grouped by category across all non-deleted records.

Response:

```
{
  "Salary": 10000.0,
  "Freelance": 5000.0,
  "Rent": 4000.0,
  "Groceries": 1500.0
}
```

GET /dashboard/recent

Returns the 5 most recently created non-deleted financial records, ordered by createdAt descending.

GET /dashboard/monthly-trend

Returns monthly aggregated income and expense data for charting and trend analysis.

Response:

```
[
  { "month": "2026-03", "income": 10000.0, "expense": 6000.0 },
  { "month": "2026-04", "income": 5000.0, "expense": 2500.0 }
]
```

5. Data Models

5.1 User Entity

Maps to the users table in MySQL.

Field	Type	Constraints	Description
id	Long	PK, Auto-increment	Primary key
name	String	NOT NULL	Full name
email	String	NOT NULL, UNIQUE	Login identifier
password	String	NOT NULL, @JsonIgnore	BCrypt hashed, never serialized
role	Role (enum)	ADMIN ANALYST VIEWER	Stored as STRING in DB
active	boolean	Default: true	Account active status
createdAt	LocalDateTime	Set on construction	Account creation timestamp

5.2 FinancialRecord Entity

Maps to the financial_records table. Uses soft delete pattern.

Field	Type	Constraints	Description
id	Long	PK, Auto-increment	Primary key
amount	Double	Required	Transaction amount
type	RecordType (enum)	INCOME EXPENSE	Stored as STRING
category	String	Required	Category label
date	LocalDate	Optional	Transaction date
description	String	Optional	Free text note
createdBy	User (ManyToOne)	FK → users.id	User who created the record
createdAt	LocalDateTime	@CreationTimestamp	Auto-set on insert
updatedAt	LocalDateTime	@UpdateTimestamp	Auto-updated on save
deleted	boolean	Default: false	Soft-delete flag

5.3 Enumerations

RecordType

- INCOME — Money received (salary, freelance, investment returns, etc.)
- EXPENSE — Money spent (rent, utilities, groceries, etc.)

Role

- ADMIN — Full access: create, view, and delete records
- ANALYST — Can create and view records, cannot delete
- VIEWER — Read-only access to all records and dashboard

6. Configuration & Deployment

6.1 Application Properties

Key configuration in `src/main/resources/application.properties`:

```
# Application
spring.application.name=finance-dashboard

# Database (credentials via environment variables)
spring.datasource.url=jdbc:mysql://localhost:3306/finance_dashboard
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.open-in-view=false

# Connection Pool (optimized for Railway free tier)
spring.datasource.hikari.maximum-pool-size=3
spring.datasource.hikari.minimum-idle=1
spring.datasource.hikari.idle-timeout=30000

# JWT
jwt.secret=${JWT_SECRET:dev-secret}
jwt.expiration=86400000

# Server
server.port=${PORT:8080}
server.forward-headers-strategy=framework

# Swagger
springdoc.swagger-ui.use-root-path=true
```


6.2 Environment Variables

The following environment variables must be set in Railway (Variables tab):

Variable	Example Value	Purpose
DB_PASSWORD	your-mysql-password	MySQL database password
JWT_SECRET	a-long-random-secret-key-32chars	JWT HMAC signing key
MYSQL_URL	jdbc:mysql://host:3306/db	Railway MySQL connection URL
PORT	8080	Auto-set by Railway

6.3 Running Locally

Prerequisites:

- Java 17 or higher
- Maven 3.6+
- MySQL 8+ running locally

Steps:

- Clone the repository: `git clone https://github.com/KunalPatil94/finance-dashboard-backend.git`
- Create a MySQL database: `CREATE DATABASE finance_dashboard;`
- Set environment variables: `DB_PASSWORD` and `JWT_SECRET`
- Run: `mvn spring-boot:run`
- Access Swagger UI: `http://localhost:8080/swagger-ui/index.html`

6.4 Deployment on Railway

- The application is deployed on Railway cloud platform
- Connected to GitHub repository (KunalPatil94/finance-dashboard-backend)
- Auto-deploys on every push to the main branch
- MySQL database provisioned as a Railway service
- Environment variables configured in Railway Variables tab
- JVM optimized for Railway free tier: `-Xms128m -Xmx256m -XX:+UseSerialGC`

7. Testing the API

7.1 Using Swagger UI

The easiest way to test all endpoints interactively:

- Navigate to: <https://resourceful-youthfulness-production-8c81.up.railway.app/swagger-ui/index.html>
- Click the Authorize button (lock icon) at the top right
- Enter your token in format: Bearer eyJhbGciOiJIUzI1NiJ9...
- All endpoints are now accessible — click any endpoint, then Try it out, then Execute

7.2 Sample cURL Workflow

Step 1 — Register a user

```
curl -X POST https://<base-url>/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Test User","email":"test@example.com","password":"Password123","role":"ANALYST"}'
```

Step 2 — Login and get token

```
curl -X POST https://<base-url>/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"Password123}"'
```

Response: { "token": "eyJhbGciOiJIUzI1NiJ9..." }

Step 3 — Create a financial record

```
curl -X POST https://<base-url>/records \
  -H "Authorization: Bearer <your-token>" \
  -H "Content-Type: application/json" \
  -d '{"amount":5000,"type":"INCOME","category":"Salary","date":"2026-04-05}"'
```

Step 4 — Get dashboard summary

```
curl -X GET https://<base-url>/dashboard/summary \
  -H "Authorization: Bearer <your-token>"
```

7.3 PowerShell (Windows)

On Windows, use `Invoke-WebRequest` with `-UseBasicParsing`:

```
Invoke-WebRequest -Uri "https://<base-url>/auth/login" \  
  -Method POST -UseBasicParsing \  
  -Headers @{ "Content-Type" = "application/json" } \  
  -Body '{"email":"test@example.com","password":"Password123"}'
```

8. Error Handling

8.1 Global Exception Handler

All `RuntimeExceptions` are caught by `GlobalExceptionHandler` (`@RestControllerAdvice`) and returned as consistent JSON:

```
{ "error": "error message here" }
```

8.2 Common Error Responses

Status	Condition	Response Body
200 OK	Request successful	Requested data or created object
400 Bad Request	Validation failure / wrong credentials	{ "error": "Invalid password" }
401 Unauthorized	Missing or expired JWT token	Empty body (Spring Security default)
403 Forbidden	Valid token but insufficient role	Empty body (Spring Security default)
404 Not Found	Record not found by ID	{ "error": "Record not found" }
500 Server Error	Unexpected server exception	Spring Boot default error response

9. Known Limitations & Future Improvements

9.1 Current Limitations

- `filterRecords()` applies only the first non-null filter parameter — combined multi-filter (e.g., type + category simultaneously) is not supported yet
- CORS is configured with `allowedOrigins("*")` — should be restricted to specific frontend domains in production
- The deleted field is named `is_deleted` in the DB column but accessed as `deleted` in JPQL — works due to Hibernate mapping but should be made consistent
- No token blacklisting — once issued, a token is valid until expiry even if user logs out
- No email uniqueness validation error handling — duplicate email returns a generic DB exception

9.2 Future Improvements

- Implement JPA Specification for multi-parameter combined filtering
- Add refresh token support with token rotation
- Add user management endpoints (list users, update role, deactivate account)
- Implement update endpoint for financial records (PUT `/records/{id}`)
- Add export functionality (PDF/CSV report of financial records)
- Add rate limiting to authentication endpoints
- Write unit and integration tests with JUnit 5 and Mockito
- Add Spring Actuator for health check and metrics endpoints

10. Project Summary

The Finance Dashboard Backend is a complete, production-deployed REST API demonstrating real-world Spring Boot backend development practices. The project implements the full stack of backend concerns including secure authentication, role-based authorization, data persistence, business logic, and cloud deployment.

The application is live at:

<https://resourceful-youthfulness-production-8c81.up.railway.app>

Swagger UI: </swagger-ui/index.html>

GitHub: github.com/KunalPatil94/finance-dashboard-backend

Developed by Kunal R. Patil | April 2026